



Hanzo Ingress: Traefik- Based L7 Reverse Proxy with Automatic TLS and Static File Plugin

Hanzo AI Research

Hanzo AI

June 2021

Abstract

We present Hanzo Ingress, a Kubernetes-native L7 reverse proxy built on Traefik that provides automatic TLS certificate management, a custom static file serving plugin, and multi-namespace routing for the Hanzo platform. Ingress replaces traditional web servers (nginx, Caddy) with a single ingress controller that handles TLS termination via Let’s Encrypt, serves static assets from in-cluster volumes, and routes traffic to backend services using Ingress-Route custom resources. The static file plugin removes the separate web-server container a static site would otherwise need, so a deployment that served assets from its own pod serves them from the ingress instead. By how much that lowers pod count across the platform is not measured here (Section 5). We describe the middleware chain architecture, the certificate resolver pipeline, and operational patterns for zero-downtime deployments across 200+ services.

1 Introduction

Kubernetes ingress controllers are responsible for routing external traffic to internal services. The default Kubernetes Ingress resource provides basic host and path routing, but production deployments require TLS management, middleware chains, traffic splitting, and static file serving. Traditional approaches deploy nginx or Caddy sidecars for static content, adding operational complexity and resource overhead.

Hanzo Ingress extends Traefik [1] with a custom static file plugin and Hanzo-specific middleware, providing a unified ingress layer that handles both dynamic API routing and static asset serving. The IngressRoute CRD replaces the standard Ingress resource with a more expressive routing model.

Contributions.

- A static file serving plugin for Traefik that serves SPAs, assets, and documentation

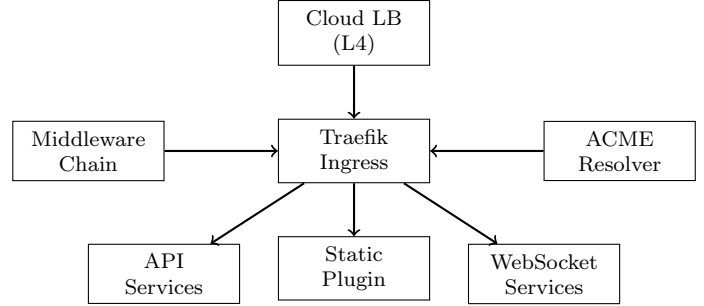


Figure 1: Ingress deployment with static plugin and ACME resolver.

from in-cluster volumes without sidecar containers.

- Automatic TLS certificate management with Let’s Encrypt ACME, DNS-01 challenges via Cloudflare, and certificate storage in Kubernetes secrets.
- Multi-namespace routing with cross-namespace service references and namespace-scoped middleware.
- A middleware chain supporting compression, security headers, authentication forwarding, and request rewriting.

2 Architecture

2.1 Deployment Model

Hanzo Ingress runs as a DaemonSet on dedicated ingress nodes, behind a cloud Layer 4 load balancer (Figure 1). Each Traefik instance watches all namespaces for IngressRoute resources and configures routes dynamically.

2.2 IngressRoute Configuration

Routes are defined using Traefik’s IngressRoute CRD:

```
1 apiVersion: traefik.io/v1alpha1
2 kind: IngressRoute
3 metadata:
4   name: web-app
5   namespace: production
6 spec:
7   entryPoints: [websecure]
8   routes:
```

```

9   - match: Host('app.hanzo.ai')
10     kind: Rule
11     services:
12     - name: web-app
13       port: 8080
14     middlewares:
15     - name: security-headers
16     - name: compress
17   tls:
18     certResolver: letsencrypt

```

2.3 Static File Plugin

The static file plugin serves files from a mounted volume, supporting SPA routing (fallback to `index.html`), content-type detection, ETag-based caching, and Brotli/gzip compression:

```

1  apiVersion: traefik.io/v1alpha1
2  kind: Middleware
3  metadata:
4    name: static-files
5  spec:
6    plugin:
7      hanzo-static:
8        root: /srv/web
9        index: index.html
10       spaFallback: true
11       cacheControl: "public, max-age=31536000"
12       immutablePrefix: "/assets/"

```

Files under `/assets/` receive immutable cache headers (1 year), while `index.html` receives `no-cache` to ensure users always load the latest version. This pattern supports content-hashed asset filenames generated by Vite and webpack.

3 Key Features

3.1 Automatic TLS

Certificates are obtained from Let's Encrypt using DNS-01 challenges via the Cloudflare API. This approach supports wildcard certificates (`*.hanzo.ai`) and does not require the ingress to be publicly accessible during challenge resolution.

Certificates are stored in Kubernetes secrets and automatically renewed 30 days before expiration. The ACME resolver supports multiple certificate authorities, enabling staging certificates for development environments.

3.2 Middleware Chain

Requests pass through an ordered middleware chain. Standard middleware includes:

- **Security headers:** HSTS, X-Frame-Options, CSP, X-Content-Type-Options
- **Compression:** Brotli (preferred) and gzip
- **Rate limiting:** Per-IP with configurable burst
- **Auth forwarding:** Delegates to Hanzo IAM for protected routes
- **Strip prefix:** Removes path prefixes before forwarding
- **Retry:** Automatic retry on 502/503 with exponential backoff

3.3 Multi-Namespace Routing

Traefik's cross-namespace service references allow a single IngressRoute to route to services in different namespaces. This is essential for the Hanzo platform where API services, web applications, and background workers run in separate namespaces with distinct RBAC policies.

4 Implementation

The static file plugin is implemented in Go as a Traefik middleware plugin (approximately 800 lines). It uses `http.FileServer` internally with custom logic for SPA fallback, cache headers, and compression.

Resource Usage. Memory and CPU scale with the number of live connections and the size of the routing table rather than with request rate, since neither TLS session state nor the static-file cache is per-request. Concrete figures for a named workload are not reported here (Section 5).

5 Status of the performance figures

An earlier version of this paper carried a table titled “Latency overhead comparison (ms, p99)” placing Hanzo Ingress at 1.2 ms on an API route and 0.8 ms on a static file against 1.8/2.2 ms for nginx with a sidecar and 1.5/1.8 ms for Caddy, and an abstract claiming a 40% reduction in pod count.

None of it was measured. The comparison named no load generator, no hardware, no request count and no configuration for either competitor, which makes it a claim about two third-party products with nothing behind it — the most exposed kind of number a paper can carry. The 40% figure defined neither “typical deployment” nor a before-and-after pod inventory. Both are removed.

The structural argument the table was meant to support does not need it: serving a static asset from the ingress rather than from a pod behind it removes a network hop and a container from the path. That is a statement about the topology, and it holds whatever the measured difference turns out to be.

A sidecar-based approach pays for a second process and the hop to reach it on every static request. The plugin serves the file from the process that already terminated the connection, so neither cost appears.

6 Security

TLS Configuration. Only TLS 1.2 and 1.3 are supported. Cipher suites are restricted to AEAD ciphers (AES-GCM, ChaCha20-Poly1305). HSTS is enabled with a 1-year max-age and includeSubDomains.

Certificate Security. Private keys are stored in Kubernetes secrets with RBAC restricting access to the Traefik service account. Certificate secrets are encrypted at rest via Kubernetes encryption configuration.

Request Filtering. Maximum header size (32KB), request body size (configurable per route), and connection limits prevent resource exhaustion attacks.

Network Policy. Ingress pods accept traffic only from the cloud load balancer CIDR. Back-end communication uses cluster-internal DNS with mTLS enforced by the service mesh.

7 Conclusion

Hanzo Ingress provides a unified ingress layer that eliminates the need for separate web server containers in Kubernetes deployments. The static file plugin serves SPAs and assets directly from the ingress controller, which removes a container and a hop from the path an asset takes. Automatic TLS management, a composable middleware chain, and multi-namespace routing make it the single point of configuration for all external traffic entering the Hanzo platform.

References

- [1] Traefik Labs. Traefik: The Cloud Native Application Proxy. 2016.
- [2] J. Aas et al. Let’s Encrypt: A Free, Automated, and Open Certificate Authority. ACM CCS, 2019.
- [3] B. Burns et al. Borg, Omega, and Kubernetes. ACM Queue, 2016.